TRƯỜNG ĐHCN VIỆT - HUNG
KHOA CÔNG NGHỆ THÔNG TIN

ĐẠI HỌC CÔNG NGHIỆP VIỆT-HUNG

CÁC LỚP ĐỐI TƯỢNG HTTPCONTEXT

TS. Nguyễn Hồng Quân

Kiến trúc tổng quan HttpContext

- Trung tâm quản lý toàn bộ thông tin về một phiên giao tiếp HTTP giữa Client và Server trong ASP.NET Core MVC



HttpContext & ASP.NET Core

Vai Trò của HttpContext

Trong ASP.NET Core, **HttpContext** chứa tất cả thông tin liên quan đến câu truy vấn HTTP hiện tại. Nó đóng vai trò là container chính bọc các đối tượng Request, Response, Session, User, Items và các dịch vụ khác.

Sự Thay Đổi Kiến Trúc

So với ASP.NET Framework cũ (System.Web), HttpContext trong Core được thiết kế lại nhẹ hơn, tối ưu hiệu năng và tách rời khỏi IIS Server. Không còn truy cập static qua HttpContext.Current mà thông qua **Dependency Injection** - DI (IHttpContextAccessor).

Các thuộc tính và lớp đối tượng chính trong HttpContext

Thuộc tính (HttpContext.)	Lớp / Interface tương ứng	Chức năng chính
Request	HttpRequest	Đại diện cho dữ liệu gửi từ Client lên: Query string, Headers, Body, Cookies, Route values, HTTP Method, Path...
Response	HttpResponse	Đại diện cho dữ liệu phản hồi trả về Client: StatusCode, Response Headers, Response Body, Response Cookies...
User	ClaimsPrincipal	Đại diện cho thông tin người dùng hiện tại sau khi xác thực (Authentication), chứa danh sách các ClaimsIdentity và Claim.
Items	IDictionary<object, object?>	Bộ lưu trữ dữ liệu dạng Key-Value tạm thời, chỉ sống trong phạm vi duy nhất của 1 Request. Dùng để chia sẻ dữ liệu giữa các Middleware hoặc Controller.
RequestServices	IServiceProvider	Container Dependency Injection (DI) có phạm vi theo Request (Scoped), dùng để giải phóng (resolve) các service cần thiết.

Các thuộc tính và lớp đối tượng chính trong HttpContext

Thuộc tính (HttpContext.)	Lớp / Interface tương ứng	Chức năng chính
Connection	ConnectionInfo	Chứa thông tin hạ tầng kết nối mạng: IP Remote/Local, Port, Client Certificate...
Session	ISession	Quản lý trạng thái phiên làm việc của Client giữa các request (yêu cầu cấu hình Session Middleware).
Features	IFeatureCollection	Tập hợp các HTTP Features do Web Server (Kestrel, IIS) hoặc Middleware hạ tầng cung cấp cho request.
WebSockets	WebSocketManager	Quản lý nâng cấp kết nối từ HTTP thông thường lên WebSocket.
TraceIdentifier	string	Chuỗi định danh duy nhất đại diện cho request đó, thường dùng để log lỗi và tracing.

Các phương thức truy cập HttpContext

- Trong Controller / Action - Sử dụng trực tiếp thuộc tính kế thừa
  - Truy cập qua this.HttpContext trong bất kỳ Action nào của Controller kế thừa từ ControllerBase.
  - Ví dụ: var user = HttpContext.User;
    var ip = HttpContext.Connection.RemoteIpAddress;
- Trong Razor View (.cshtml) - Sử dụng đối tượng @Context có sẵn
  - Sử dụng trực tiếp thuộc tính @Context trong file Razor view .cshtml.Ví dụ:
    @Context.Request.Path
    @Context.User.Identity.Name
- Trong Custom Services / Class - Inject qua IHttpContextAccessor
  - Tầng Service/Repository không kế thừa Controller nên cần Inject IHttpContextAccessor qua DI.
  - Cần đăng ký trong Program.cs: builder.Services.AddHttpContextAccessor();

Luồng HttpContext trong Middleware Pipeline

- Sơ đồ Luồng Di chuyển HTTP Request & Response
- Client Request ➔ Kestrel Server ➔ Middleware 1 (Logging) ➔ Middleware 2 (Auth) ➔ Routing / Controller Action
- Client Response ← Kestrel Server ← Middleware 1 (OnCompleted) ← Middleware 2 (OnStarting) ← Action Result Executed
  - HttpResponse.OnStarting: Chạy ngay trước khi HTTP Headers được ghi ra Network Stream. Đây là NƠI CUỐI CÙNG có thể sửa Status Code hoặc Headers!
  - HttpResponse.OnCompleted: Chạy sau khi toàn bộ Response Body đã gửi xong về Client. Thích hợp dọn dẹp tài nguyên hoặc ghi Audit Log.

HTTP Request



HTTP Request (HttpRequest)

- Đọc Dữ Liệu Vào
  - Truy cập Tham số Query String (Request.Query), Dữ liệu Form POST (Request.Form), và thông tin Header (Request.Headers).
- Cookies & Routing
  - Đọc Cookie được gửi từ Browser qua Request.Cookies và lấy thông tin Route parameter thông qua Request.RouteValues.
- Async Body Stream
  - Hỗ trợ đọc dữ liệu Body bất đồng bộ (JSON API payload, File Upload IFormFile) giúp tối ưu bộ nhớ RAM tối đa.

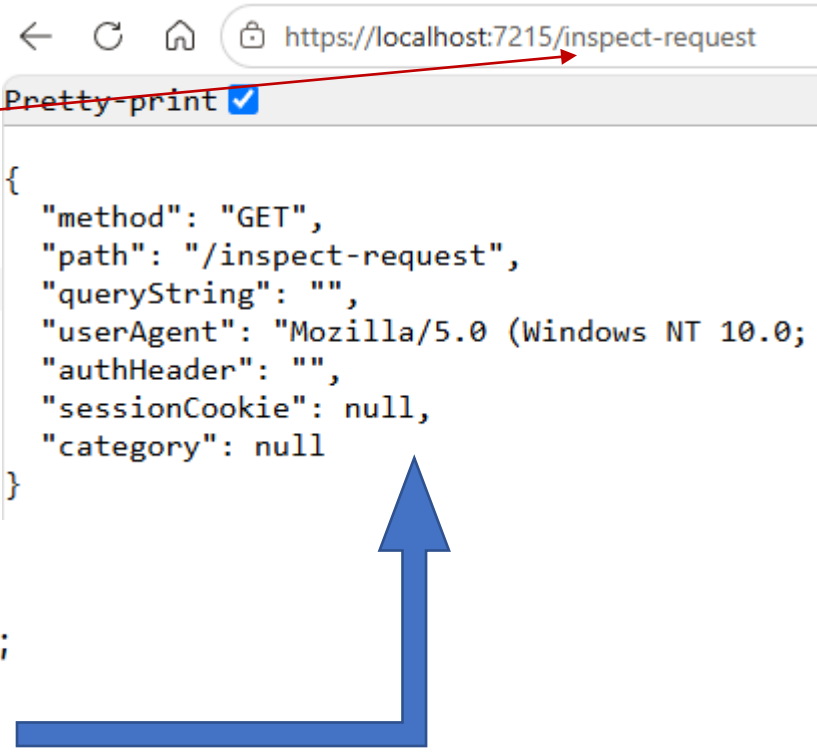
HTTP Request (HttpRequest)

- Các Thuộc tính:
  - Request.Method: Phương thức HTTP gửi lên (GET, POST, PUT, DELETE, PATCH, OPTIONS.)
  - Request.Scheme & Host: Scheme (http/https) và Host (ví dụ: localhost:5001 hoặc api.mysite.com).
  - Request.Path & QueryString: Path chứa đường dẫn tài nguyên (/api/products), QueryString chứa tham số URL (?cat=2&page=1).
  - Request.Headers: Tập hợp Header gửi từ Client (User-Agent, Authorization, Accept, Content-Type). Truy cập dạng Dictionary: Request.Headers["HeaderName"].
  - Request.Cookies: Đọc các Cookie được trình duyệt tự động đính kèm trong Request (IRequestCookieCollection).
  - Request.IsHttps: Giá trị bool xác định kết nối có được mã hóa qua SSL/TLS hay không.

HTTP Request (HttpRequest)

❖Demo đọc HTTP Request trong Controller

```
[HttpGet("inspect-request")]
0 references
public IActionResult InspectRequest()
{
    // 1. Đọc thông tin cơ bản
    string method = HttpContext.Request.Method;
    string path = HttpContext.Request.Path;
    string? queryString = HttpContext.Request.QueryString.Value;
    // 2. Đọc Headers & Cookies
    string userAgent = HttpContext.Request.Headers["User-Agent"].ToString();
    string authHeader = HttpContext.Request.Headers["Authorization"].ToString();
    string? sessionCookie = HttpContext.Request.Cookies["ASP.NET_SessionId"];
    // 3. Đọc Query Parameter
    string? category = HttpContext.Request.Query["category"];
    return Ok(new
    {
        Method = method,Path = path, QueryString = queryString,
        UserAgent = userAgent, AuthHeader = authHeader,
        SessionCookie = sessionCookie,Category = category
    });
}
```



- Chú ý! **OkObjectResult** là một Action Result được dùng để trả về phản hồi HTTP Status Code 200 OK kèm theo dữ liệu (payload) ở phần thân (body) của response với 3 nhiệm vụ chính:
  - ***Xác nhận thành công (Status 200)***: Thông báo cho Client (Frontend, Postman, App mobile...) biết yêu cầu đã được xử lý hoàn tất thành công.
  - ***Đóng gói dữ liệu***: Nhận bất kỳ đối tượng C# nào (Class, DTO, List, String, Anonymous Object...) để gửi về cho Client
  - ***Tự động Serialize***: ASP.NET Core sẽ tự động chuyển đổi đối tượng C# đó thành định dạng dữ liệu thích hợp (mặc định là JSON, hoặc XML tùy vào cấu hình Content Negotiation).

HTTP Request (HttpRequest)

❖Xử lý Form Data, Stream Body & File Upload

Form Data (Request.Form) - Dữ liệu Form Submit

- Đọc dữ liệu POST từ Form (application/x-www-form-urlencoded).
- Ví dụ: string username = Request.Form["txtUser"];
- Chỉ truy cập được khi Request có Content-Type tương ứng.

File Upload (Request.Form.Files) - Tải tệp đính kèm

- Đọc danh sách File upload (multipart/form-data) dạng IFormFileCollection.
- Mỗi file là IFormFile cung cấp FileName, Length, ContentType.
- Lưu file bất đồng bộ: await file.CopyToAsync(stream);

Request Body Stream - Đọc Payload thô (JSON/XML)

- Stream đọc trực tiếp dữ liệu thô từ Client (Request.Body).
- Mặc định là Forward-only Stream (chỉ đọc được 1 lần).
- Muốn đọc nhiều lần cần gọi: HttpContext.Request.EnableBuffering();

Http Response



HttpResponse - Phản hồi dữ liệu về Client

- HttpResponse trong ASP.NET Core đại diện cho phản hồi HTTP (HTTP Response) mà Server chuẩn bị gửi về cho Client. Các chức năng chính:
  - ***Response.StatusCode***: Mã trạng thái HTTP gửi về Client (200 OK, 400 BadRequest, 401 Unauthorized, 404 NotFound, 500 ServerError). Mặc định là 200.
  - Response.ContentType: Mô tả định dạng Response Body (text/html; charset=utf-8, application/json, application/pdf).
  - Response.Headers: Tập hợp Header phản hồi về Client (Cache-Control, Set-Cookie, X-Frame-Options, Access-Control-Allow-Origin).
  - Response.Cookies: Thêm Cookie mới về Browser bằng Response.Cookies.Append("key", "val", options) hoặc xóa bằng Response.Cookies.Delete("key").
  - Response.Body Stream: Stream đầu ra ghi dữ liệu trực tiếp về Client (Response.Body).
  - Response.HasStarted: Thuộc tính bool rất quan trọng. Trả về true nếu HTTP Headers đã bắt đầu được gửi qua mạng.

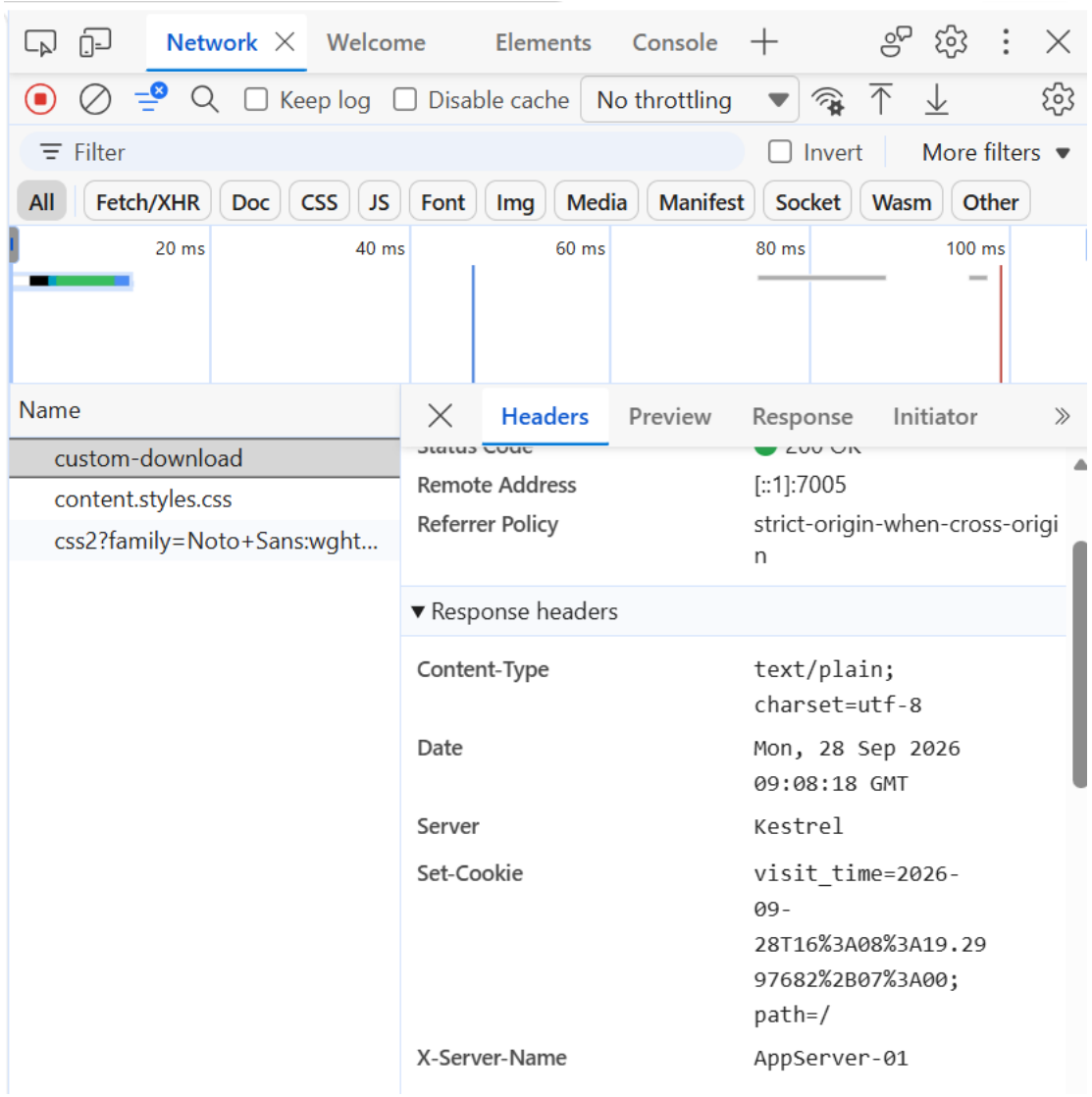
HttpResponse - Phản hồi dữ liệu về Client

```
namespace TH05_TestApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class DemoController : ControllerBase
    {
        [HttpGet("custom-download")]
        0 references
        public async Task CustomResponseExample()
        {
            // 1. Đặt Status Code
            Response.StatusCode = StatusCodes.Status200OK;
            // 2. Thiết lập Header và ContentType
            Response.ContentType = "text/plain; charset=utf-8";
            Response.Headers.Append("X-Server-Name", "AppServer-01");
            // 3. Ghi Cookie xuống Client
            Response.Cookies.Append("visit_time", DateTime.Now.ToString("O"));
            // 4. Ghi trực tiếp nội dung vào Response Body
            await Response.WriteAsync("Dữ liệu này được ghi trực tiếp bằng HttpResponse!");
        }
    }
}
```

- Ví dụ:
  - B1: Tạo dự án ASP.NET Core Web API, tạo file DemoController.cs trong thư mục Controllers) và dán đoạn code sau:

HttpResponse - Phản hồi dữ liệu về Client

- Ví dụ:
  - B2: Chạy project (API project không có giao diện GUI)
  - B3: Gọi API và kiểm tra kết quả (chú ý hiệu chỉnh cổng cho phù hợp): ***https://localhost:7005/api/demo/custom-download***
  - Kiểm tra kết quả dùng Developer Tools (F12) → Network → Nhấn F5. Hoặc dùng phần mềm Postman / Insomnia



HttpResponse - Phản hồi dữ liệu về Client

❖Khi nào dùng HttpResponse thay vì IActionResult (Ok(), BadRequest())?

	Dùng IActionResult (Ok(), File(), Json())	**Dùng trực tiếp HttpResponse**
Bản chất	Hướng đối tượng cao cấp, sạch mã (clean code), tự xử lý serialization.	Thao tác cấp thấp (low-level), tương tác trực tiếp với luồng dữ liệu HTTP.
Trường hợp sử dụng	Hầu hết các API Controller thông thường.	Lập trình **Middleware**, ghi file dữ liệu cực lớn (streaming), xử lý Server-Sent Events (SSE).
Khả năng kiểm thử	Dễ dàng viết Unit Test cho kết quả trả về.	Khó viết Unit Test hơn do phụ thuộc trực tiếp vào HttpContext.

Ghi trực tiếp Response Stream & Redirect

- Direct Stream Writing - Ghi trực tiếp dữ liệu thô
  - Sử dụng await Response.WriteAsync("Hello World") để ghi chuỗi HTML/Text.
  - Ghi chuỗi JSON hoặc file Binary trực tiếp vào Response.Body stream.
  - Hữu ích khi viết Middleware hoặc streaming file lớn.
- URL Redirect - Điều hướng trình duyệt
  - Response.Redirect("/login"): Chuyển hướng tạm thời (302 Found).
  - Response.Redirect("/new-url", permanent: true): Chuyển hướng vĩnh viễn (301 Moved Permanently).
  - Trình duyệt sẽ tự động gọi Request mới đến URL chỉ định.
- Quy tắc HasStarted - Tránh lỗi Runtime Exception
  - Khi Response.HasStarted == true, Headers đã được gửi đi.
  - Nếu cố gắng thay đổi StatusCode hoặc Headers sau thời điểm này, ASP.NET Core sẽ báo lỗi InvalidOperationException!
  - LUÔN kiểm tra if (!Response.HasStarted) trước khi thay đổi Header.

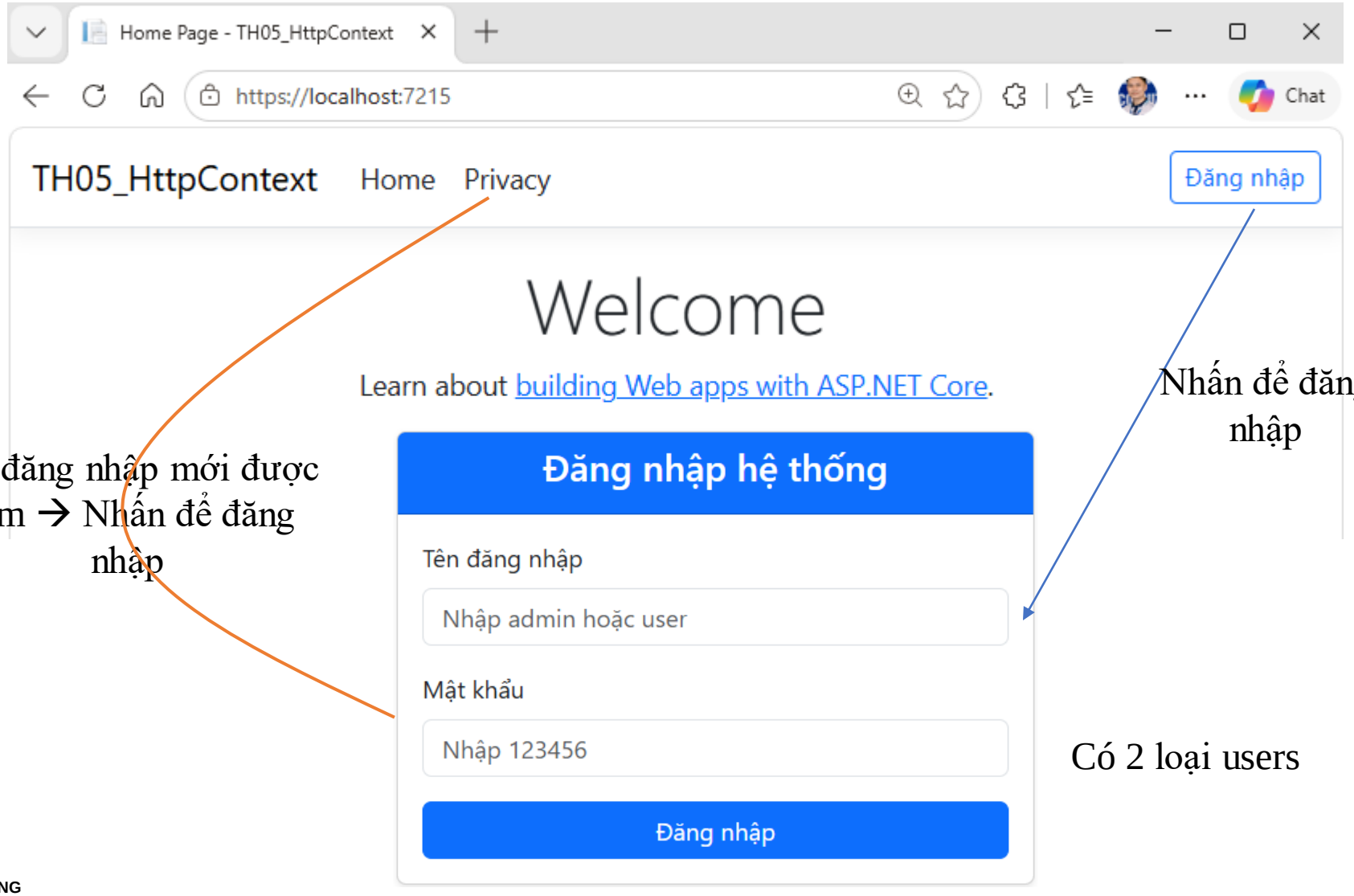
Quản lý Nhận dạng người dùng (HttpContext.User)



Kiến trúc Bảo mật Claims-Based trong ASP.NET Core

- ClaimsPrincipal (HttpContext.User)
  - Đại diện cho chủ thể người dùng hiện tại đang thực hiện Request.
- ClaimsIdentity:
  - Một người dùng có thể sở hữu một hoặc nhiều Nhận dạng (ví dụ: Identity từ Cookie, Identity từ Bearer Token).
- Claim:
  - Một mẩu thông tin xác thực dạng Key-Value mô tả người dùng (ví dụ: NameIdentifier, Email, DateOfBirth, Department).
- User.Identity.IsAuthenticated:
  - Kiểm tra người dùng đã đăng nhập thành công hay chưa (true/false).
- User.IsInRole("Admin"):
  - Kiểm tra nhanh người dùng có thuộc vai trò (Role) chỉ định hay không.
- User.FindFirst(ClaimTypes.Email):
  - Trích xuất giá trị Claim cụ thể từ danh sách Claims của User.

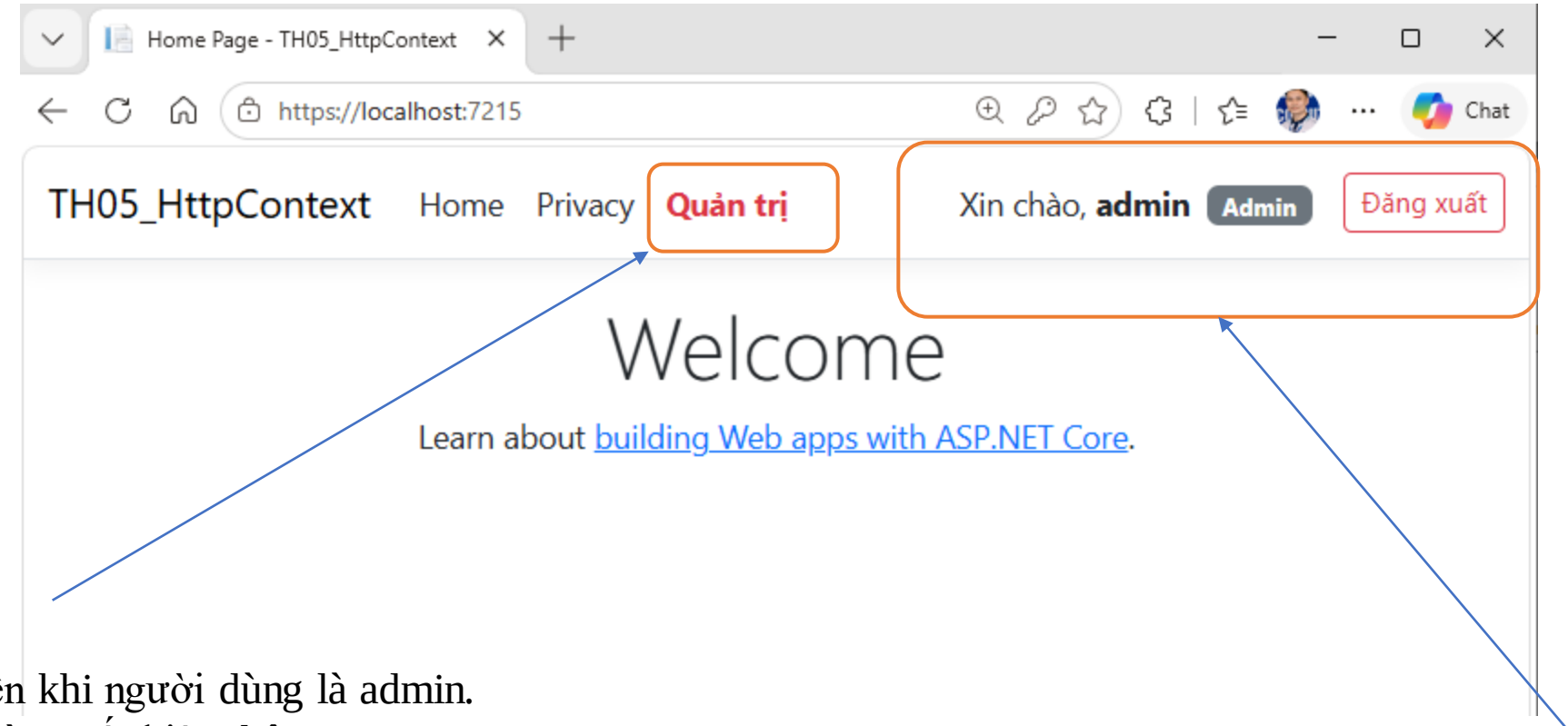
Ví dụ: Truy xuất Claims & Phân quyền User



Chỉ đăng nhập mới được xem → Nhấn để đăng nhập

Nhấn để đăng nhập

Có 2 loại users



Xuất hiện khi người dùng là admin.
Nhấn vào xuất hiện thông tin sau

{"userId":null,"email":"admin@viu.edu.vn","role":"Admin","isAdmin":true}

Xuất hiện sau khi đăng nhập

{"userId":null,"email":"admin
@viu.edu.vn","role":"Admin"
,"isAdmin":true}

```
[Authorize]
[HttpGet]
0 references
public IActionResult GetUserProfile()
{
    // 1. Kiểm tra trạng thái đăng nhập
    if (!User.Identity.IsAuthenticated)
        return Unauthorized();
    // 2. Trích xuất Claims
    string? userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    string? email = User.FindFirst(ClaimTypes.Email)?.Value;
    string? role = User.FindFirst(ClaimTypes.Role)?.Value;
    // 3. Kiểm tra Phân quyền
    bool isAdmin = User.IsInRole("Admin");
    return Ok(new
    {
        UserId = userId,
        Email = email,
        Role = role,
        IsAdmin = isAdmin
    });
}
```

Thứ tự thực hiện

- Cấu hình Cookie Authentication trong Program.cs
- Tạo Model cho Đăng nhập (LoginViewModel.cs)
- Tạo AccountController.cs xử lý Đăng nhập / Đăng xuất
- Áp dụng Phân quyền trong Controller
  - Sử dụng thuộc tính [Authorize], [Authorize(Roles = "Admin")], [AllowAnonymous]
- Phân quyền hiển thị giao diện trong Razor View (_Layout.cshtml)
  - Kiểm tra trạng thái đăng nhập và Role của user trực tiếp trên View bằng User.Identity và User.IsInRole()
- Tạo view Login.cshtml và AccessDenied.cshtml
- Hiển thị thông tin người dùng

Quản lý Trạng thái Session & Cookies



HttpContext.Session

- HttpContext.Session - Bộ nhớ Phiên làm việc trên Server
  - Lưu trữ dữ liệu tạm thời gắn liền với phiên làm việc của từng User.
  - Yêu cầu cấu hình Session Middleware trong Program.cs:
    builder.Services.AddSession();
    app.UseSession();
  - Lưu/Đọc dữ liệu: Session.SetString("key", "val") / Session.GetString("key").
  - Nên giới hạn dung lượng để tránh quá tải RAM Server.

Cookies

- Cookies (Request / Response) - Lưu trữ phía Client Browser
  - Lưu trữ chuỗi text nhỏ trực tiếp ở Browser của Client.
  - Đọc Cookie từ Request: Request.Cookies["CartId"];
  - Ghi Cookie mới qua Response: Response.Cookies.Append("UserPref", "Dark");
  - Cần chú ý bảo mật: CookieOptions.HttpOnly = true, Secure = true, SameSite = SameSiteMode.Lax.

Các Đối tượng Truyền dữ liệu từ Controller sang View

- ViewData: Dictionary dạng [string, object].
- ViewBag: Wrapper dạng dynamic bao bọc
- ViewData.TempData: Lưu trữ dữ liệu ngắn hạn qua một HTTP Request (thường dùng thông báo).
- Strongly Typed View: Truyền trực tiếp đối tượng Model (Khuyến nghị).